

large amounts of event data.

Real-time monitoring challenges:

Collecting and pushing data to InfluxDB in real time required an efficient strategy to avoid performance bottlenecks. To achieve this, metrics were gathered at fixed intervals (every second) and transmitted asynchronously. This non-blocking approach ensured that monitoring tasks did not interfere with the core application processes.

Error handling and system

resilience: The system needed to handle a range of potential failures, including network interruptions, database crashes, and malformed input. These issues were addressed through robust error-handling mechanisms that provide clear, actionable messages and ensure continued system operation even under adverse conditions.

The modular design of this contemporary EMS makes the system extensible, and comprehensive error

handling enhances its reliability. Future development will target improving the system by incorporating functionalities

such as event notifications, multiple user roles, and support for more advanced analytics. **END** 

References

- Chen, Y., Zhang, H., & Liu, M. (2020), A Prometheus-Grafana based monitoring system for Kubernetes clusters, *Journal of Cloud Computing*, 9(1), 22–33
- Singh, R., & Kumar, P. (2019), API usage monitoring for improved service availability, *International Journal of Web Services Research*, 16(3), 45–58
- Albrecht, J., Suresh, A., & Wang, L. (2018), Distributed tracing in microservice architectures: Limitations and future directions, *ACM Computing Surveys*, 51(4), 72–88
- Li, Q., & Zhao, X. (2021), Anomaly detection in cloud-native environments using unsupervised learning, *IEEE Transactions on Network and Service Management*, 18(2), 1500–1513
- Mehta, A., & Rao, S. (2020), Telemetry collection and analysis in hybrid cloud infrastructures, *Proceedings of the 2020 IEEE International Conference on Cloud Computing*, 211–218
- Pereira, F., Mendes, R., & Costa, P. (2022), Log preprocessing and pattern mining for early error detection in distributed systems, *Journal of Systems and Software*, 185, 111229

 **By: Maturi Msnvs Raghunadh, Kolipakula Charan Sri Sai, Kaku Chokshi Malyadri Yadav, Kushal Venkata Sai Prasad Dasari, and B. Thangaraju**

The authors are associated with the Open Source Technology Lab at the International Institute of Technology, Bengaluru.

■ ■ Continue from page... 73

Limitation of statistical vs logical pattern matching:

BERT is a statistical pattern matcher, and it can misinterpret negation or sarcasm if the training data is not diverse enough. For example, a BERT model cannot interpret ‘The screen is anything but small’.

Computational cost: A fine-tuned BERT system usually needs a GPU (like an NVIDIA). Training on a standard laptop CPU is slow.

A pre-training model: As BERT is a pre-trained masked language model (guessing hidden words), ABSA often faces challenges in its classification tasks (since sequential classification is used in ABSA).

Domain incompatibility: The standard BERT models are generally trained on Wikipedia and books; that’s why it often faces challenges for texts with slang, emojis or

technical jargon.

Sentiment analysis using the BERT mechanism is a simple and elegant approach. It offers an easy way to understand the contextual meaning of a sentence rather than understanding the words in isolation or from left-to-right. While BERT has enhanced NLP and is the engine behind modern ABSA, it has several imitations also. As BERT provides a statistical framework for NLP, it is necessary to be concerned about the construction of the sentence as well as the logic of the constructions. **END** 

 **By: Dipankar Ray**

The author has more than 30 years of experience in software development. He has published two books on MATLAB and medical image processing (published by PHI Learning and SPD).